

EduGenie: Google Gemini Powered Learning Assistant

Project Description

EduGenie is a lightweight AI-powered educational assistant that simplifies learning through generative AI. Designed for students of all academic levels, EduGenie enables users to:

- Ask questions and receive smart, concise answers
- Understand complex concepts through simplified explanations
- Generate quizzes from topics or text
- Receive personalized learning recommendations
- Summarize large educational passages

Built with **FastAPI** for the backend and a simple **HTML+CSS frontend**, EduGenie leverages lightweight and cloud-based AI models for local efficiency and cloud power. It works well on devices like the Mac M1, making it accessible to a broad range of learners and developers.

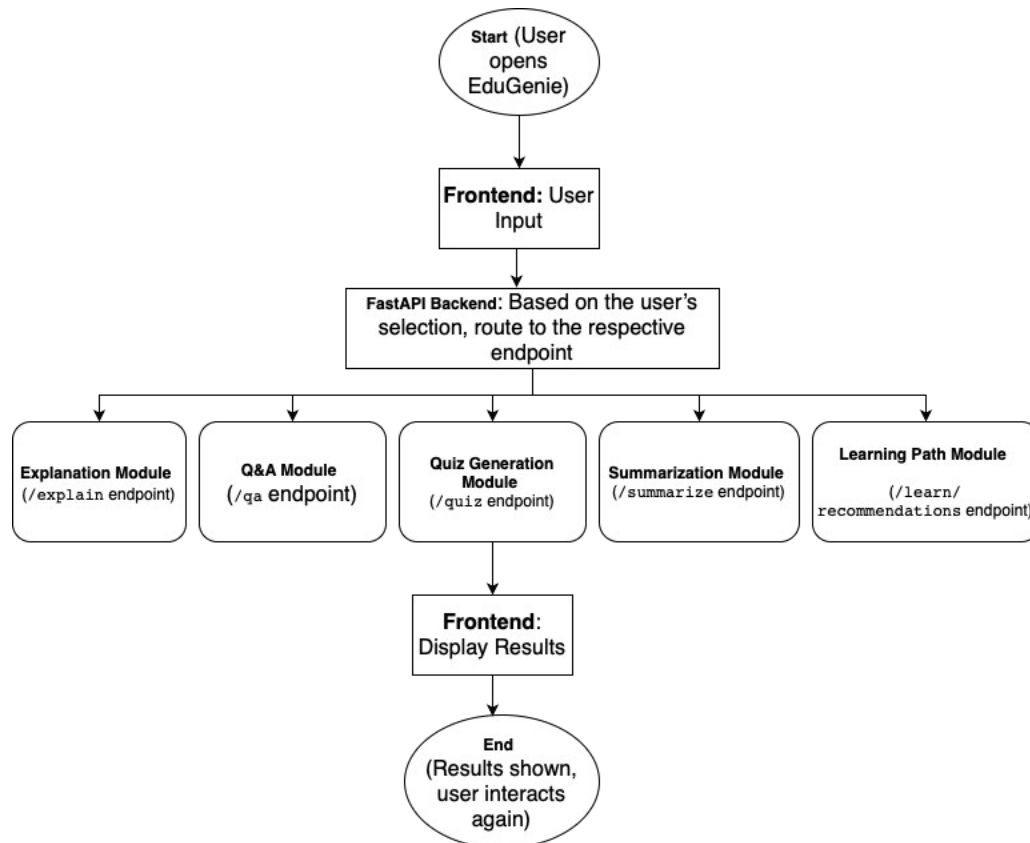
Scenarios

Scenario 1: A student wants to know about oceans and rivers uses EduGenie to ask “Which is the largest ocean?”

Scenario 2: A student wants to know the level of her understanding of “The Pythagoras Theorem” and clicks “Generate Quiz.”

Scenario 3: A learner exploring SQL requests a learning path which is a structured plan with beginner to advanced topics, timelines, and suggestions.

ARCHITECTURAL DIAGRAM



Pre-requisites

1. Python 3.10+

- Official Documentation: [Google AI for Developers](#)
- Installation Guide: [Tom's Hardware](#)
- Popular Tutorial: [Python Programming Tutorial - Full Course for Beginners](#)
- Installation Steps:
 1. Download the latest Python 3.10+ installer for your operating system from the official website.
 2. Run the installer and ensure you check the box that says "Add Python to PATH".
 3. Follow the installation prompts to complete the setup.
 4. Verify the installation by opening a terminal or command prompt and typing `python --version`.

2. FastAPI Framework –

- Official Documentation: [FastAPI](#)
 - User Guide: [FastAPI](#)
 - Popular Tutorial: [FastAPI Crash Course](#)
3. HTML & CSS – Basic templating used in /templates and /static
4. Google Gemini API Key
- Official Documentation: [Google AI for Developers](#)
 - Setup Guide: [GeeksforGeeks](#)
 - Popular Tutorial: [How to Use Google Gemini API Key](#)
 - Setup Steps:
 1. Visit the Google AI Studio and sign in with your Google account.
 2. Create a new project and enable the Gemini API.
 3. Generate an API key and securely store it.
5. Uvicorn (ASGI Server)
- Official Documentation: [PyPI](#)
 - Installation Steps:
 1. Install Uvicorn using pip: `pip install uvicorn`
6. Jinja2 (HTML Templating Engine)
- Official Documentation: [Jinja2 Documentation](#)
 - Installation Steps:
 1. Install Jinja2 using pip: `pip install jinja2`

Project Workflow

MILESTONE 1: Model Selection and Architecture

Activity 1.1: Select AI Models

Google Gemini (via the google-genai SDK, default model gemini-flash-latest):

- Used for: Q&A, summarization, quiz generation, learning paths and explanation fallback
- Benefits: advanced reasoning, structured JSON output (JSON mode) and fast cloud inference with a free tier
- Reliability: if the model is overloaded (503 “high demand”) or rate limited (429), EduGenie retries with exponential backoff and then falls back to backup models (gemini-flash-latest, gemini-flash-lite-latest)
- Configurable: the model can be changed in .env (GEMINI_MODEL, for example gemini-3.5-flash)

LaMini-Flan-T5-783M (local, optional):

- Used for: concept explanation
- Benefits: instruction-tuned, lightweight and CPU-compatible; EduGenie automatically uses Gemini if it is not installed

Activity 1.2: Folder Architecture

```
EduGenie/
├─ main.py           # FastAPI app and API endpoints
├─ gemini_client.py  # Gemini client (retry + fallback)
├─ explanation_module.py  # Concept explanation logic
├─ qna.py            # Question answering
├─ quiz_module.py    # Quiz generation and JSON validation
├─ summary_module.py # Summarization
├─ learning_path.py  # Learning recommendations
├─ templates/index.html # HTML frontend
├─ static/style.css  # Styling
└─ static/script.js  # Frontend logic
```

— tests/	# Automated tests (pytest) and load test
— requirements.txt	# Python dependencies
— .env.example	# Configuration template (API key, model)

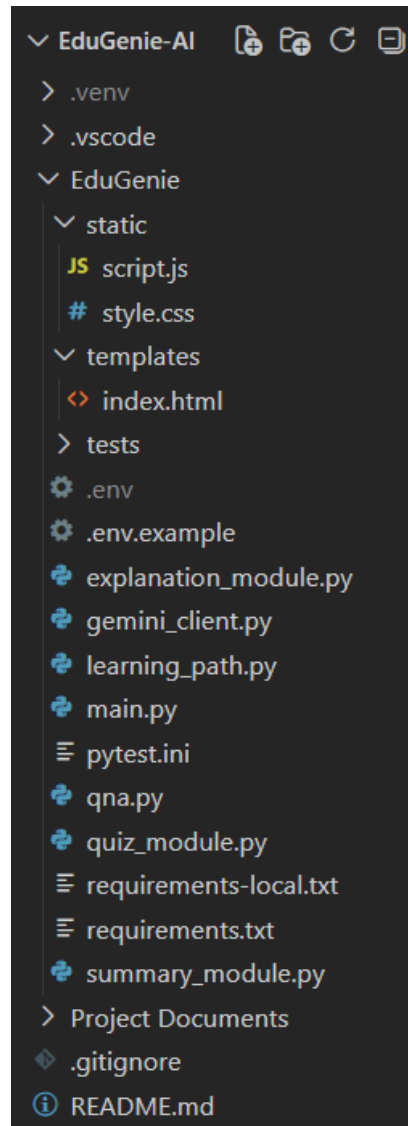


FIG. Showing the Folder Architecture

MILESTONE 2: Core Functionalities Development

Activity 2.1: Module Implementation

Gemini Client

All AI features share one function, `generate()` in `gemini_client.py`. It loads the API key from `.env`, sends the prompt to the configured Gemini model (optionally in JSON mode), retries temporary errors such as 503 “high demand” or 429 rate limits with exponential backoff, tries the backup

models when the main model stays busy, and converts SDK errors into clear messages for the user.

```
57
58 def generate(prompt: str, *, json_mode: bool = False, temperature: float | None = None) -> str:
59     """Send `prompt` to Gemini and return the response text.
60
61     `json_mode=True` asks Gemini to reply with raw JSON (no Markdown).
62     Raises GeminiNotConfiguredError / GeminiError on failure.
63     """
64     api_key = get_api_key()
65     if api_key is None:
66         raise GeminiNotConfiguredError(
67             "Gemini API key is not configured. Copy EduGenie/.env.example to "
68             "EduGenie/.env and set GEMINI_API_KEY, then restart the server."
69         )
70
71     from google.genai import types
72
73     config = types.GenerateContentConfig(
74         temperature=temperature,
75         response_mime_type="application/json" if json_mode else None,
76     )
77
78     try:
79         response = _client(api_key).models.generate_content(
80             model=get_model_name(),
81             contents=prompt,
82             config=config,
83         )
84     except Exception as exc: # network errors, invalid key, quota, unknown model ...
85         logger.error("Gemini request failed: %s", exc)
```

Fig. Gemini client with retry and fallback

Explanation Module

EduGenie uses the LaMini-Flan-T5-783M model, a lightweight yet capable generative model, to explain concepts in a simplified and highly readable way for school students. The model is loaded once and generates a short, clear explanation on the CPU. When the local model is not installed, or if it fails, `explain_topic()` automatically falls back to Gemini, so the feature always works. This makes EduGenie a reliable study companion for beginners and self-learners.

```
def get_backend() -> str:
    backend = os.getenv("EXPLAIN_BACKEND", "auto").strip().lower()
    return backend if backend in {"auto", "local", "gemini"} else "auto"

def local_model_available() -> bool:
    """True when the optional transformers/torch dependencies are installed."""
    return all(importlib.util.find_spec(m) is not None for m in ("transformers", "torch"))

def _load_local_model():
    """Load LaMini-Flan-T5 once (thread-safe). Downloads it on first use."""
    global _explain_tokenizer, _explain_model
    if _explain_model is None:
        with _model_lock:
            if _explain_model is None:
                from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

                logger.info("Loading local explanation model %s (first run downloads it)...", LOCAL_MODEL_NAME)
                _explain_tokenizer = AutoTokenizer.from_pretrained(LOCAL_MODEL_NAME)
                _explain_model = AutoModelForSeq2SeqLM.from_pretrained(LOCAL_MODEL_NAME)
                _explain_model.eval()
                logger.info("Local explanation model loaded.")
    return _explain_tokenizer, _explain_model

81 def explain_with_gemini(topic: str) -> str:
82     return gemini_client.generate(EXPLAIN_PROMPT.format(topic=topic), temperature=0.5)
83
84
85 def explain_topic(topic: str) -> tuple[str, str]:
86     """Return (explanation, source) where source is "local" or "gemini".
87
88     Raises gemini_client.GeminiError if every available backend fails.
89     """
90     topic = topic.strip()
91     backend = get_backend()
92
93     if backend == "local" or (backend == "auto" and local_model_available()):
94         try:
95             explanation = explain_with_local_model(topic)
96             if explanation:
97                 return explanation, "local"
98         except Exception:
99             logger.exception("Local explanation model failed; falling back to Gemini")
100
101     return explain_with_gemini(topic), "gemini"
```

Fig. Explanation module logic

QnA Module

The Q&A module uses Google Gemini to answer a wide range of academic and general-knowledge questions. A focused prompt asks Gemini to answer clearly and concisely in simple language, to give a small example when it helps and not to invent facts.

```
import gemini_client

QNA_PROMPT = """You are EduGenie, a friendly and accurate AI tutor for students.
Answer the student's question clearly and concisely (a short paragraph or a few bullet
points). Use simple language, give a small example when it helps, and do not invent facts.
If the question is not educational or is unclear, politely say so and suggest how to rephrase.

Question: {question}
"""

def answer_question_with_gemini(question: str) -> str:
    """Return a concise answer to `question`. Raises gemini_client.GeminiError on failure."""
    return gemini_client.generate(QNA_PROMPT.format(question=question.strip()), temperature=0.3)
```

Fig. Q/A Module

Quiz Module

EduGenie generates multiple-choice questions (three by default, up to ten) from a topic or passage, each with four options, the correct answer and a short explanation. Gemini is asked to reply in JSON mode so the output is structured for easy integration.

```
17
18 QUIZ_PROMPT = """You are a quiz generator for students.
19
20 From the following input, create {n} multiple-choice questions. The input is either a
21 passage (then only ask about information in the passage) or a short topic name (then ask
22 about the core concepts of that topic). Each question must include:
23 - A "question"
24 - A list of exactly 4 "options" (plausible distractors, no "All of the above")
25 - A correct "answer" that must exactly match one of the options
26 - A one-sentence "explanation" of why the answer is correct
27
28 Format your output as valid JSON only, like this:
29 [
30     {{
31         "question": "What is ...?",
32         "options": ["A", "B", "C", "D"],
33         "answer": "A",
34         "explanation": "Because ..."
35     }}
36 ]
37
38 Input:
39 \"\"\"
40 {text}
41 \"\"\"
42 """
```

```

48
49 def clean_json_block(text: str) -> str:
50     """Remove Markdown ```json code fences (if any) around the model output."""
51     text = text.strip()
52     match = re.search(r"```(?:json)?\s*\n?(.*?)```", text, flags=re.DOTALL)
53     if match:
54         text = match.group(1).strip()
55     return text
56

```

Fig. Quiz Module Logic

The response is cleaned with `clean_json_block()`, which removes any Markdown code fences, and then validated by `parse_quiz()`: every question must have exactly four options and an answer that matches one of them (letter answers such as "B" are mapped to the option text). Invalid questions are discarded, and if no valid question remains a clear error message is returned instead of crashing.

```

58 def parse_quiz(raw: str) -> list[dict]:
59     """Parse and validate the model output. Raises QuizFormatError if it is unusable."""
60     cleaned = clean_json_block(raw)
61     try:
62         data = json.loads(cleaned)
63     except json.JSONDecodeError:
64         # Fall back to the first JSON array found anywhere in the text.
65         match = re.search(r"\[.*\]", cleaned, flags=re.DOTALL)
66         if not match:
67             raise QuizFormatError("The AI response was not valid JSON. Please try again.")
68         try:
69             data = json.loads(match.group(0))
70         except json.JSONDecodeError as exc:
71             raise QuizFormatError("The AI response was not valid JSON. Please try again.") from exc
72
73     if isinstance(data, dict): # e.g. {"questions": [...]}
74         data = data.get("questions") or data.get("quiz") or []
75     if not isinstance(data, list):
76         raise QuizFormatError("The AI response did not contain a list of questions.")
77
78     quiz: list[dict] = []
79     for item in data:
80         if not isinstance(item, dict):
81             continue
82         question = str(item.get("question", "")).strip()
83         options = item.get("options")
84         answer = str(item.get("answer", "")).strip()
85         if not question or not isinstance(options, list) or len(options) != 4:

```

```

117
118 def generate_quiz(text: str, num_questions: int = DEFAULT_NUM_QUESTIONS) -> list[dict]:
119     """Generate a validated MCQ quiz from `text`. Raises gemini_client.GeminiError on failure."""
120     n = max(1, min(int(num_questions), MAX_NUM_QUESTIONS))
121     raw = gemini_client.generate(QUIZ_PROMPT.format(n=n, text=text.strip()), json_mode=True, temperature=0.4)
122     return parse_quiz(raw)[:n]
123

```

Fig. Quiz validation logic

Summary Module

This feature uses Gemini to summarize long paragraphs into a one-sentence overview followed by key bullet points, which is ideal for quick revision. The prompt keeps the core facts, removes repetition and forbids adding information that is not in the text.

```
5 import gemini_client
6
7 SUMMARY_PROMPT = """Summarize the following educational text in simple language for quick revision.
8 - Start with a one-sentence overview.
9 - Then list the key points as short bullet points.
10 - Keep all important facts, remove repetition, and do not add information that is not in the text.
11
12 Text:
13 \{"\"
14 {text}
15 \{"\"
16 """
17
18
19 def summarize_text(text: str) -> str:
20     """Return a concise summary of `text`. Raises gemini_client.GeminiError on failure."""
21     return gemini_client.generate(SUMMARY_PROMPT.format(text=text.strip()), temperature=0.2)
22
```

Fig. Summary Module

Learning Path Module

The `get_learning_recommendations()` function uses Google Gemini to generate a personalized, structured learning path for any topic, adapted to the learner's level (beginner, intermediate or advanced). It organizes the key topics by difficulty with time estimates, recommends free resources such as documentation, videos and books, suggests practice projects and ends with a week-by-week timeline.

```
5 import gemini_client
6
7 LEVELS = ("beginner", "intermediate", "advanced")
8
9 LEARNING_PATH_PROMPT = """You are an expert AI tutor. The student wants to learn about: {topic}.
10 The student's current level is: {level}.
11
12 Create a structured and adaptive learning path formatted in Markdown:
13 1. A short introduction to what the student will be able to do at the end.
14 2. Sections "## Beginner", "## Intermediate" and "## Advanced" (start from the student's level;
15 | briefly list earlier-level prerequisites if they are skipping ahead).
16 3. In every section list the key topics in the order they should be learned, each with a
17 | one-line description and an estimated time (for example "~3 days").
18 4. A "## Resources" section with free, well-known resources (official documentation,
19 | YouTube channels, articles, books). Only recommend resources you are confident exist.
20 5. A "## Practice Projects" section with 2-3 hands-on project ideas.
21 6. A "## Suggested Timeline" section summarising a realistic week-by-week plan.
22 """
23
24
25 def get_learning_recommendations(topic: str, level: str = "beginner") -> str:
26     """Return a Markdown learning path for `topic`. Raises gemini_client.GeminiError on failure."""
27     level = level.lower().strip() if level else "beginner"
28     if level not in LEVELS:
29         level = "beginner"
30     prompt = LEARNING_PATH_PROMPT.format(topic=topic.strip(), level=level)
31     return gemini_client.generate(prompt, temperature=0.5)
32
```

Fig. Learning Path Module

Activity 2.2: Backend API with FastAPI

Defined RESTful endpoints for each module:

- **GET /qa** – question answering
- **POST /explain** – concept explanation
- **POST /quiz** – quiz generation
- **POST /summarize** – summarization
- **GET /learn/recommendations** – learning path
- **GET /health** – configuration status (plus interactive API docs at /docs)

Each endpoint validates the input (empty or too long input returns HTTP 400), calls its module and returns JSON. If Gemini is still busy after the automatic retries the API returns HTTP 503 with a friendly message; other AI errors return HTTP 502.

```
111 # Q&A - GET API using Gemini
112 @app.get("/qa", tags=["learning"])
113 def answer_question(question: str = Query(..., description="The question to answer")):
114     question = validate_input(question, "a question", MAX_TOPIC_CHARS * 4)
115     if isinstance(question, JSONResponse):
116         return question
117     try:
118         return {"question": question, "answer": answer_question_with_gemini(question)}
119     except gemini_client.GeminiError as exc:
120         return ai_error(exc)
121
122
123 # Explanation - POST API (local LaMini-Flan-T5 or Gemini)
124 @app.post("/explain", tags=["learning"])
125 def explain_api(body: ExplainRequest):
126     topic = validate_input(body.topic, "a topic", MAX_TOPIC_CHARS)
127     if isinstance(topic, JSONResponse):
128         return topic
129     try:
130         explanation, source = explain_topic(topic)
131         return {"topic": topic, "explanation": explanation, "source": source}
132     except gemini_client.GeminiError as exc:
133         return ai_error(exc)
134
135
136 # Summarization - POST API
137 @app.post("/summarize", tags=["learning"])
138 def summarize_api(body: TextRequest):
139     text = validate_input(body.text, "text to summarize", MAX_TEXT_CHARS)
140     if isinstance(text, JSONResponse):
141         return text
142     try:
143         return {"summary": summarize_text(text)}
144     except gemini_client.GeminiError as exc:
145         return ai_error(exc)
```

Fig. Endpoints of API

```
148 # Quiz Generation - POST API
149 @app.post("/quiz", tags=["learning"])
150 def quiz_api(body: QuizRequest):
151     text = validate_input(body.text, "text or a topic for the quiz", MAX_TEXT_CHARS)
152     if isinstance(text, JSONResponse):
153         return text
154     try:
155         quiz = generate_quiz(text, body.num_questions)
156         logger.info("Generated quiz with %d questions", len(quiz))
157         return {"quiz": quiz}
158     except gemini_client.GeminiError as exc:
159         return ai_error(exc)
160
161
162 # Learning Recommendations - GET API
163 @app.get("/learn/recommendations", tags=["learning"])
164 def learning_recommendation_api(
165     topic: str = Query(..., description="Topic the student wants to learn"),
166     level: str = Query("beginner", description="beginner | intermediate | advanced"),
167 ):
168     topic = validate_input(topic, "a topic", MAX_TOPIC_CHARS)
169     if isinstance(topic, JSONResponse):
170         return topic
171     try:
172         return {"topic": topic, "level": level, "recommendation": get_learning_recommendations(topic, level)}
173     except gemini_client.GeminiError as exc:
174         return ai_error(exc)
```

Fig. Endpoints of API

MILESTONE 3: Frontend Development

Activity 3.1: Build Web Interface

HTML Form (index.html):

- Task dropdown (Ask a Question, Explain a Concept, Generate a Quiz, Summarize a Passage, Recommend a Learning Path)
- Text area for user input with a character counter and example suggestions
- Options for the number of quiz questions and the learner's current level
- Submit and Clear buttons, and a result card with a Copy button

CSS Styling (style.css):

- Responsive design for mobile, tablet and desktop, with automatic dark mode
- Styled buttons, input areas, result container and quiz options (green for correct, red for wrong)

```

index.html X .env
EduGenie > templates > index.html > ...
2  <html lang="en">
10 <body>
11  <header class="hero">
15  </header>
16
17  <main class="container">
18    <form id="taskForm" class="card" novalidate>
19      <label for="task"><strong>What would you like to do?</strong></label>
20      <select id="task" name="task">
21        <option value="qa"> ? Ask a Question (Q&A)</option>
22        <option value="explain"> ? Explain a Concept</option>
23        <option value="quiz"> ? Generate a Quiz</option>
24        <option value="summarize"> ? Summarize a Passage</option>
25        <option value="path"> ? Recommend a Learning Path</option>
26      </select>
27
28      <label for="userInput" id="inputLabel"><strong>Your question</strong></label>
29      <textarea id="userInput" name="userInput" rows="5" maxlength="20000" required
30        | placeholder="Which is the largest ocean?"></textarea>
31      <div class="hint-row">
32        <span id="hint" class="hint">Ask any academic or general-knowledge question.</span>
33        <span id="charCount" class="hint">0 / 20000</span>
34      </div>
35
36      <div class="options-row">
37        <div id="quizOptions" class="option hidden">
38          <label for="numQuestions">Number of questions</label>
39          <select id="numQuestions">
40            <option value="3" selected>3</option>
41            <option value="5">5</option>
42            <option value="10">10</option>
43          </select>
44        </div>
  
```

```

45         <div id="pathOptions" class="option hidden">
46             <label for="level">Your current level</label>
47             <select id="level">
48                 {% for level in levels %}
49                 <option value="{{ level }}">{{ level | capitalize }}</option>
50                 {% endfor %}
51             </select>
52         </div>
53     </div>
54
55     <div class="examples">
56         <span class="hint">Try:</span>
57         <div id="examples" class="chips"></div>
58     </div>
59
60     <div class="actions">
61         <button type="submit" id="submitBtn">Get Answer</button>
62         <button type="button" id="clearBtn" class="secondary">Clear</button>
63     </div>
64 </form>

```

Fig. Showing Frontend Integration

Activity 3.2: Live Integration

- Form submission sends a GET/POST request (fetch API in script.js) to the matching FastAPI endpoint
- A loading indicator is shown while the AI works; if Gemini is busy it shows that EduGenie is retrying automatically
- The result appears below the input box in real time, formatted for easy reading
- Quiz questions are clickable: the answer is checked instantly and a running score is shown
- Errors are shown in a message box with a Try again button when retrying can help

MILESTONE 4: Deployment

Activity 4.1: Run Locally

```

python -m venv .venv
.venv\Scripts\Activate.ps1      # Windows
source .venv/bin/activate       # macOS / Linux
pip install -r EduGenie/requirements.txt
# copy EduGenie/.env.example to EduGenie/.env and set GEMINI_API_KEY
cd EduGenie
uvicorn main:app --reload

```

- Navigate to: <http://127.0.0.1:8000>

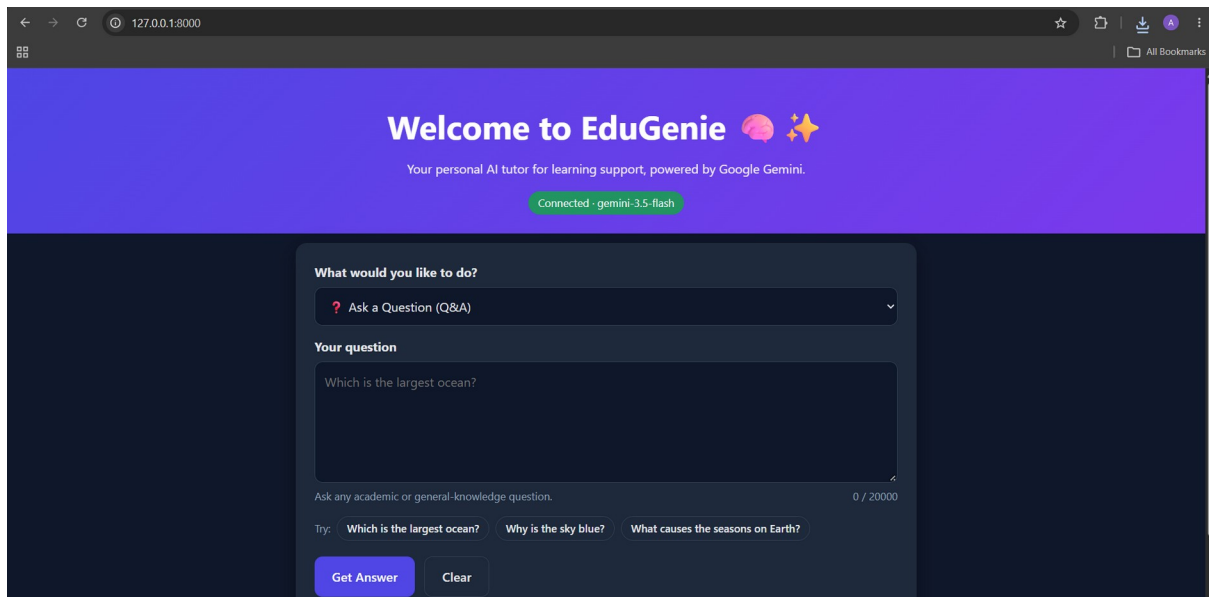


Fig. Showing the Application can be accessed

Activity 4.2: Functional Testing

- Ask a question
- Get an explanation
- Generate a quiz
- Summarize content
- Get a personalized learning plan

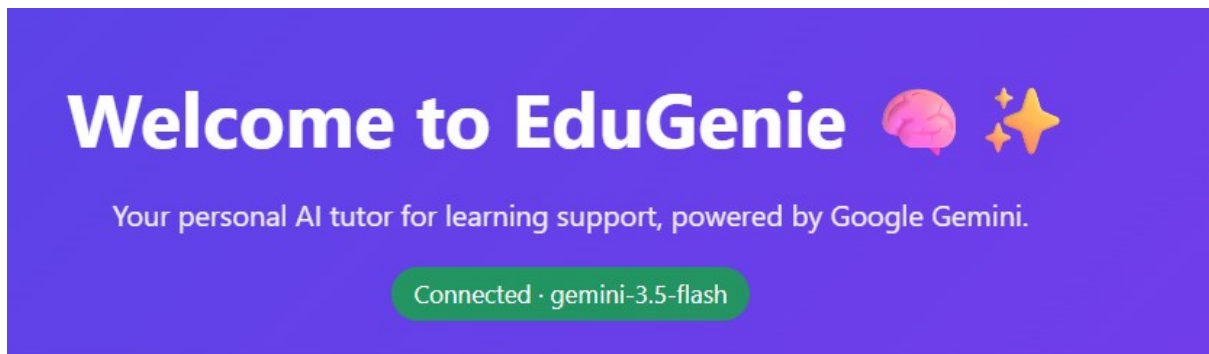


Fig. EDUGENIE

What would you like to do?

? Ask a Question (Q&A)

? Ask a Question (Q&A)

💡 Explain a Concept

📄 Generate a Quiz

📄 Summarize a Passage

📖 Recommend a Learning Path

Ask any academic or general-knowledge question.
0 / 2000

Try:

Which is the largest ocean?

Why is the sky blue?

What causes the seasons on Earth?

Get Answer

Clear

Fig. EDUGENIE

Automated tests: running `pytest -v` inside the EduGenie folder executes the test suite, which checks every endpoint, input validation, quiz parsing and the retry/fallback behaviour without needing an API key.

```
(.venv) PS C:\Users\Lenovo\OneDrive\Desktop\Github\EduGenie-AI> pytest -v
EduGenie/tests/test_api.py::test_static_files_served PASSED
EduGenie/tests/test_api.py::test_health PASSED
EduGenie/tests/test_api.py::test_qa PASSED
EduGenie/tests/test_api.py::test_qa_requires_question PASSED
EduGenie/tests/test_api.py::test_explain PASSED
EduGenie/tests/test_api.py::test_explain_trailing_slash_like_original_doc PASSED
EduGenie/tests/test_api.py::test_explain_empty_topic PASSED
EduGenie/tests/test_api.py::test_explain_topic_too_long PASSED
EduGenie/tests/test_api.py::test_summarize PASSED
EduGenie/tests/test_api.py::test_summarize_empty PASSED
EduGenie/tests/test_api.py::test_quiz PASSED
EduGenie/tests/test_api.py::test_quiz_num_questions PASSED
EduGenie/tests/test_api.py::test_quiz_invalid_num_questions PASSED
EduGenie/tests/test_api.py::test_quiz_invalid_ai_output PASSED
EduGenie/tests/test_api.py::test_quiz_empty PASSED
EduGenie/tests/test_api.py::test_learning_recommendations PASSED
EduGenie/tests/test_api.py::test_learning_recommendations_requires_topic PASSED
EduGenie/tests/test_api.py::test_gemini_failure_returns_502 PASSED
EduGenie/tests/test_api.py::test_missing_api_key_returns_503 PASSED
```

Fig. Automated test results

Exploring EduGenie:

Asking questions:

Select “Ask a Question”, type a question such as “Which is the largest ocean?” and click Get Answer.

What would you like to do?

? Ask a Question (Q&A) ▾

Your question

Which is the largest ocean?

Ask any academic or general-knowledge question. 27 / 20000

Try: Which is the largest ocean? Why is the sky blue? What causes the seasons on Earth?

Get Answer **Clear**

Answer Gemini Copy

Hello there! I would be happy to help you with that.

The largest ocean on Earth is the **Pacific Ocean**.

It is absolutely massive! To give you an idea of its size, the Pacific Ocean covers more than 30% of the Earth's surface, which means it is actually larger than all of the Earth's continents put together.

Explanation of any topic:

Select “Explain a Concept” and enter a topic such as Photosynthesis. The badge shows whether the local LaMini-Flan-T5 model or Gemini produced the explanation.

What would you like to do?

💡 Explain a Concept

Concept or topic

Gravity

Enter a concept to get a simple, student-friendly explanation.

7 / 20000

Try: [Photosynthesis](#) [Gravity](#) [Recursion in programming](#)

Explain

Clear

Explanation

Gemini

Copy

Gravity is an invisible force. It pulls objects toward each other. It is what keeps our feet on the ground.

Imagine a trampoline. If you place a heavy bowling ball in the middle, it makes a deep dip. If you roll a small marble onto the trampoline, it rolls down into the dip toward the heavy ball. Earth is like that heavy bowling ball. It bends space and pulls everything toward it.

For example, when you drop an apple, it does not float away. It falls straight to the floor because Earth's gravity is pulling it down. The bigger an object is, the stronger its pull. Since Earth is huge, it has a very strong pull on us.

Key takeaway: Gravity is the invisible glue that holds our whole world together.

Summarising long paragraphs:

Paste a long passage, select "Summarize a Passage" and click Summarize to get a one-line overview and key bullet points.

What would you like to do?

Summarize a Passage

Text to summarize

The water cycle describes how water evaporates from the surface of the earth, rises into the atmosphere, cools and condenses into clouds, and falls again to the surface as precipitation. Water falling on land collects in rivers and lakes, soil, and porous layers of rock, and much of it flows back into the oceans, where it will once more evaporate. The cycling of water in and out of the atmosphere is a significant aspect of the weather patterns on Earth.

Paste long educational content to get a concise revision summary. 457 / 20000

Try: The water cycle describes how water e...

Summarize Clear

Summary Gemini Copy

Overview The water cycle is the continuous movement of water between the Earth's surface and the atmosphere, which plays a major role in shaping Earth's weather patterns.

Key Points

- **Evaporation and Condensation:** Water evaporates from the Earth's surface, rises into the atmosphere, and cools to form clouds.
- **Precipitation:** Water falls back to the Earth's surface as precipitation.
- **Collection:** On land, water collects in rivers, lakes, soil, and porous rock layers.
- **Return to Oceans:** Much of this water flows back into the oceans, where the evaporation process starts again.
- **Weather Impact:** The movement of water in and out of the atmosphere is a significant part of Earth's weather.

Generating quizzes:

It generates three questions (or 5 or 10) with 4 options each. Clicking an option checks the answer instantly: a correct choice turns green, a wrong choice turns red and the correct answer is highlighted with a short explanation and a running score.

What would you like to do?

 Generate a Quiz

Topic or passage

The Pythagoras Theorem

Enter a topic or paste a passage to generate multiple-choice questions.

22 / 20000

Number of questions

3

Try: [The Pythagoras Theorem](#) [Solar System](#) [World War II](#)

Generate Quiz

Clear

Quiz

[Gemini](#) [Copy](#)

1. What is the standard algebraic formula for the Pythagorean theorem, where 'c' represents the length of the hypotenuse and 'a' and 'b' represent the lengths of the other two sides?

A) $a + b = c$

B) $a^2 + b^2 = c^2$

C) $a^2 - b^2 = c^2$

D) $a^2 + c^2 = b^2$

✓ Correct! — The Pythagorean theorem states that in a right-angled triangle, the square of the hypotenuse is equal to the sum of the squares of the other two sides.

2. To which specific type of triangle does the Pythagorean theorem exclusively apply?

A) Equilateral triangle

B) Obtuse triangle

C) Right-angled triangle

D) Acute triangle

✓ Correct! — The Pythagorean theorem is a fundamental relation in Euclidean geometry that only applies to triangles containing a 90-degree angle.

3. Which of the following sets of side lengths represents a valid Pythagorean triple?

A) 3, 4, 6

B) 5, 12, 13

C) 6, 8, 11

D) 1, 2, 3

✓ Correct! — The numbers 5, 12, and 13 form a Pythagorean triple because 5 squared (25) plus 12 squared (144) equals 13 squared (169).

Score: 3 / 3 🎉 Quiz complete!

Guiding with learning recommendations from beginners level to advanced level:

Enter a topic such as SQL, choose the current level and click Get Learning Path.

What would you like to do?

Recommend a Learning Path

What do you want to learn?

Machine Learning

Get a structured beginner-to-advanced plan with timelines and resources. 16 / 20000

Your current level

Beginner

Try: SQL Machine Learning Web Development

Get Learning Path

Clear

Learning Path

Gemini Copy

Welcome to your Machine Learning (ML) journey!

By the end of this learning path, you will transition from a complete beginner to someone who can **clean raw data, build predictive models (such as predicting prices or classifying images), evaluate their performance, and deploy them to the cloud.** You will understand both the mathematical intuition behind AI and how to write clean, production-ready ML code.

Beginner

Since you are starting at the beginner level, we will begin with the absolute essentials: programming tools, data manipulation, and foundational algorithms.

- Topic 1: Python for Data Science (NumPy & Pandas)**

Description: Learn how to manipulate multi-dimensional arrays and clean tabular datasets using Python's industry-standard libraries. *Estimated Time: ~5 days*
- Topic 2: Data Visualization (Matplotlib & Seaborn)**

Description: Learn to plot graphs, histograms, and correlation matrices to understand your data before feeding it to an algorithm. *Estimated Time: ~3 days*
- Topic 3: Introduction to ML Concepts**

Description: Understand the core differences between Supervised, Unsupervised, and Reinforcement Learning, as well as train/test splits. *Estimated Time: ~2 days*

- **Topic 4: Linear & Logistic Regression**

Description: Master the foundational algorithms used for predicting continuous values (regression) and binary categories (classification). *Estimated Time:* ~5 days

- **Topic 5: Decision Trees & Random Forests**

Description: Learn how tree-based models make decisions and how combining multiple trees (ensembling) prevents errors. *Estimated Time:* ~5 days

- **Topic 6: Model Evaluation Metrics**

Description: Learn how to measure your model's success using metrics like Accuracy, Precision, Recall, F1 Score, and Mean Squared Error (MSE). *Estimated Time:* ~3 days

Intermediate

Prerequisites: Comfortable with Python, basic algebra, and Scikit-Learn basics.

- **Topic 1: Feature Engineering & Data Preprocessing**

Description: Learn how to handle missing data, encode categorical variables, scale features, and select the most important variables. *Estimated Time:* ~5 days

- **Topic 2: Support Vector Machines (SVM) & K-Nearest Neighbors (KNN)**

Description: Explore distance-based and boundary-based classification algorithms for non-linear datasets. *Estimated Time:* ~4 days

- **Topic 3: Unsupervised Learning (K-Means & PCA)**

Description: Group unlabeled data into clusters and reduce the dimensionality of massive datasets without losing key information. *Estimated Time:* ~5 days

- **Topic 4: Gradient Boosting (XGBoost, LightGBM)**

Description: Learn the state-of-the-art ensemble techniques that dominate tabular data competitions on Kaggle. *Estimated Time:* ~5 days

- **Topic 5: Cross-Validation & Hyperparameter Tuning**

Description: Use GridSearch and RandomSearch alongside K-Fold cross-validation to find the absolute best settings for your models. *Estimated Time:* ~4 days

Advanced

Prerequisites: Strong grasp of intermediate ML algorithms, calculus/linear algebra basics, and model tuning.

- **Topic 1: Introduction to Neural Networks & PyTorch**

Description: Understand artificial neurons, activation functions, backpropagation, and how to build simple deep learning networks. *Estimated Time:* ~7 days

- **Topic 2: Convolutional Neural Networks (CNNs) for Computer Vision**

Description: Learn how to process, analyze, and classify image data using deep learning. *Estimated Time:* ~6 days

- **Topic 3: Recurrent Neural Networks (RNNs) & Transformers**

Description: Explore sequence models used for Natural Language Processing (NLP), text generation, and translation. *Estimated Time:* ~8 days

- **Topic 4: Model Deployment & MLOps Basics**

Description: Learn how to package your trained model into an API using FastAPI and deploy it to a cloud platform (like Hugging Face Spaces or AWS). *Estimated Time:* ~6 days

Resources

These are highly respected, completely free resources to guide your learning:

- **Interactive Coding:** [Kaggle Learn](#) (Short, hands-on micro-courses covering Python, Pandas, and Machine Learning).
- **Video Lectures:** [StatQuest with Josh Starmer \(YouTube\)](#) (The absolute best visual and intuitive explanations of ML math and algorithms).
- **Core Course:** [Andrew Ng's Machine Learning Specialization on Coursera](#) (Can be audited for free; the gold standard for ML intuition).
- **Deep Learning:** [Fast.ai - Practical Deep Learning for Coders](#) (A top-tier, free, code-first approach to neural networks).
- **Documentation:** [Scikit-Learn Official User Guide](#) (Incredibly well-written with practical code examples).

Practice Projects

Project 1 (Beginner): House Price Predictor

- **Goal:** Predict the sale price of houses based on features like square footage, bedrooms, and location.
- **What you will do:** Use the classic *Ames Housing Dataset* (available on Kaggle). Clean the data, handle missing values, and train a **Random Forest Regressor** using Scikit-Learn. Evaluate your model using Root Mean Squared Error (RMSE).

Project 2 (Intermediate): Customer Segmentation

- **Goal:** Group an e-commerce company's customer base into distinct marketing personas.
- **What you will do:** Use an online retail dataset. Perform feature engineering to calculate Recency, Frequency, and Monetary (RFM) values for each customer. Apply **K-Means Clustering** and use the Elbow Method to find the optimal number of customer segments.

Project 3 (Advanced): Image Classification API

- **Goal:** Build a deep learning model that classifies images and serve it via a web API.
- **What you will do:** Train a **Convolutional Neural Network (CNN)** using PyTorch to classify images of

Suggested Timeline

This 12-week plan assumes roughly **10–12 hours of study per week**.

- **Weeks 1-2: Foundations of Data**
 - Focus on Python, NumPy, Pandas, and exploratory data analysis (EDA).
- **Weeks 3-4: Beginner Algorithms**
 - Learn Linear/Logistic Regression, Decision Trees, and model evaluation metrics.
 - **Milestone:** Complete **Project 1 (House Price Predictor)**.
- **Weeks 5-6: Intermediate Data Prep & Clustering**
 - Dive into feature engineering, scaling, and Unsupervised Learning (K-Means).
 - **Milestone:** Complete **Project 2 (Customer Segmentation)**.
- **Weeks 7-8: Advanced Tabular Models & Tuning**
 - Master XGBoost, Cross-Validation, and Hyperparameter tuning.
- **Weeks 9-10: Deep Learning Foundations**
 - Learn PyTorch basics, Neural Networks, and build your first CNN for image classification.
- **Weeks 11-12: Deployment & Final Polish**
 - Learn FastAPI, deploy your model, and finalize your portfolio.
 - **Milestone:** Complete **Project 3 (Image Classification API)**.

It provides details of resources to learn from, along with stepwise guidance throughout, from the beginner level to the advanced level.

Conclusion:

EduGenie has been developed as an accessible AI-powered learning assistant that brings together cloud-based intelligence from Google Gemini and a simple, low-footprint web interface. It answers questions instantly, explains complex concepts in simple language, generates interactive quizzes with instant feedback, summarizes long passages and builds personalized learning paths from beginner to advanced level. Because the heavy AI processing happens in the cloud, and an optional local model is available for explanations, learners with modest hardware can still access high-quality, personalized learning support.

During development the project reached its key milestones in AI integration, prompt engineering, API design, frontend development and testing. Each feature lives in its own module with a carefully designed prompt, the FastAPI backend validates every request, and an automated test suite verifies all endpoints and error cases. The modular design keeps the code easy to read, extend and maintain.

The project also involved real challenges. AI output had to be reliable, so quizzes use Gemini's JSON mode together with strict validation of every question. The Gemini API can be temporarily overloaded ("high demand") or rate limited, so EduGenie retries automatically with exponential backoff, falls back to backup models and otherwise shows a clear message with a Try again button. The API key is kept in a private .env file instead of the source code, and AI responses are safely rendered in the browser. These challenges provided valuable learning in prompt engineering, error handling, cloud API integration and user-centred design.

Looking ahead, EduGenie has strong potential for growth: voice-based interaction for hands-free and accessible learning, multilingual support, a mobile application, user accounts with progress-tracking dashboards, gamification such as badges and learning streaks, integration with learning management systems like Moodle and Google Classroom, and image or PDF input for snapshot-based doubt solving.

In conclusion, EduGenie shows how generative AI can make learning more personal, interactive and inclusive. It is a solid foundation for a smart, scalable and learner-first study companion.

Submitted by:

1. Arumparithi B (Team Leader)

2. Atchayabharath K S

Team ID: SWTID-2026-7859

Mentor:

Date: 03 October 2026